

Chapter 4

User Login and JWT Generation

Verifying user credentials and issuing a signed JSON Web Token — the foundation of stateless authentication.

4.1 Introduction

After implementing user registration, users can now create accounts in the system. However, creating an account is only the first step. The application must also provide a secure way for users to prove their identity whenever they want to access their account.

This process is called **Authentication**. Authentication answers one simple question:

“Is this user really who they claim to be?”

When a user attempts to log in, they provide their email and password. The backend verifies these credentials against the data stored in the database. If the credentials are correct, the backend generates a **JSON Web Token (JWT)** and returns it to the client.

Unlike traditional web applications, our backend does **not** create a server-side session. Instead, it relies on JWT-based authentication, making the application stateless and easier to scale.

4.2 What is Authentication?

Authentication is the process of verifying a user's identity. For example, suppose Rahul enters the following credentials:

Email: rahul@example.com
Password: myPassword123

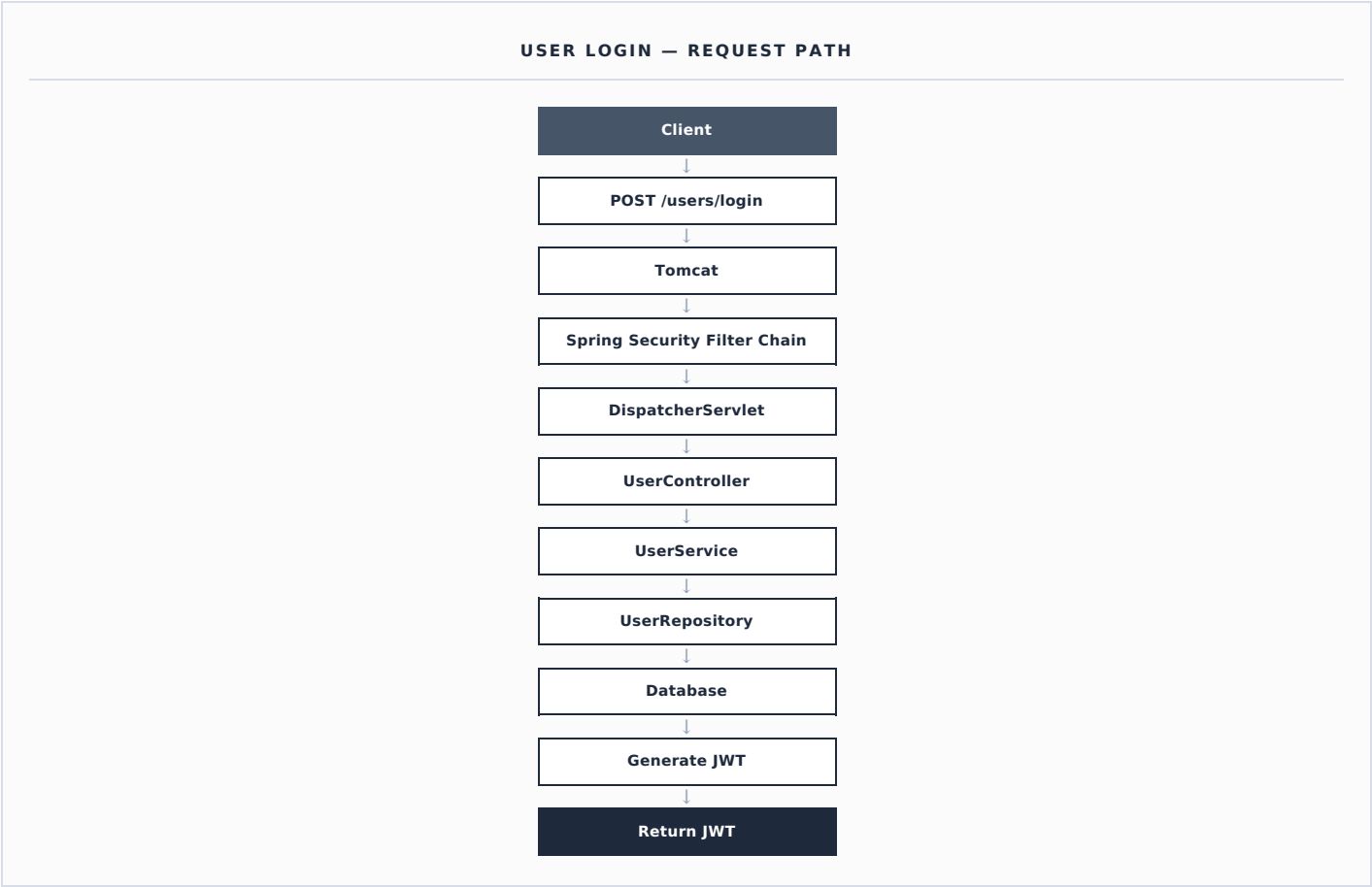
The backend does not trust these values immediately. Instead, it performs two checks:

- Does a user with this email exist?
- Does the entered password match the encrypted password stored in the database?

Only if both conditions are true is the user considered authenticated. Authentication does **not** determine what the user is allowed to do — it only proves who the user is.

4.3 Login Request Flow

The complete login request follows the same layered architecture used during registration. Instead of saving a new user, the service now verifies credentials and generates a token.



The Ten-Step Login Sequence

1

Client Sends Login Request

WHAT HAPPENS

The client sends an HTTP POST request to the login endpoint:

```
{
  "email": "rahul@example.com",
  "password": "myPassword123"
}
```

The client is requesting authentication.

WHY

The backend needs the user's credentials before it can verify identity. Without these credentials, authentication is impossible.

2

Request Reaches Tomcat

WHAT HAPPENS

Tomcat receives the HTTP request and creates `HttpServletRequest` and `HttpServletResponse` . The request is then forwarded into the Spring framework.

WHY

Tomcat acts as the web server. It accepts incoming requests and hands them over to Spring Boot.

3

Spring Security Filter Chain

WHAT HAPPENS

Every request passes through the Spring Security Filter Chain. Since this is the login endpoint, the request usually does not contain a JWT. Our custom `JwtFilter` checks for the Authorization header, finds no Bearer token, and simply forwards the request.

WHY

A user cannot provide a JWT before logging in because the JWT has not been generated yet. Login endpoints are therefore configured using:

```
.requestMatchers("/users/**").permitAll()
```

This allows unauthenticated users to reach the login controller.

4

DispatcherServlet

WHAT HAPPENS

The DispatcherServlet examines the request URL, identifies the matching controller method responsible for handling `/users/login` , and forwards the request to `UserController` .

WHY

The DispatcherServlet is responsible for routing every incoming request to the correct controller.

5 UserController

WHAT HAPPENS

The controller receives the login request, converts the incoming JSON into a Java object, and calls the service:

```
@PostMapping("/login")
public String login(@RequestBody User user) {
    return userService.loginUser(user);
}
```

WHY

The controller should not verify passwords or generate JWTs. Its only responsibility is handling HTTP communication — business logic belongs in the service layer.

6 UserService

WHAT HAPPENS

The service performs the authentication process:

- Find the user by email
- Verify that the password matches
- Generate a JWT
- Return the JWT

The service makes all authentication decisions.

WHY

Authentication is business logic. Keeping it inside the service allows multiple controllers to reuse the same authentication process.

7 Find User by Email

WHAT HAPPENS

```
User user = userRepository.findByEmail(email);
```

If no user is found, an exception is thrown.

WHY

Passwords should never be checked before confirming that the user actually exists. The email uniquely identifies a user.

8 Verify Password

WHAT HAPPENS

The entered password is compared with the encrypted password stored in the database:

```
passwordEncoder.matches(rawPassword, hashedPassword);
```

Notice that the password is **not decrypted** — BCrypt hashes the entered password and compares it with the stored hash.

WHY

BCrypt is a one-way hashing algorithm. One-way hashes cannot be reversed, which makes password storage much more secure.

9 Generate JWT

WHAT HAPPENS

If both email and password are correct, the service requests a JWT from `JwtService`. The token contains claims about the authenticated user and is digitally signed using the application's secret key:

```
Subject    → rahul@example.com
Role       → DONOR
Issued At   → Current Time
Expiration  → Future Time
```

WHY

Instead of remembering authenticated users through server sessions, the backend allows the client to remember its own authentication information using a signed JWT — keeping the backend stateless.

10 Return JWT

WHAT HAPPENS

The generated JWT is returned to the controller, which sends the token back to the client:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

The client stores this token; future requests include it inside the Authorization header.

WHY

The JWT acts as proof that the user has already been authenticated. The user does not need to enter their password for every request.

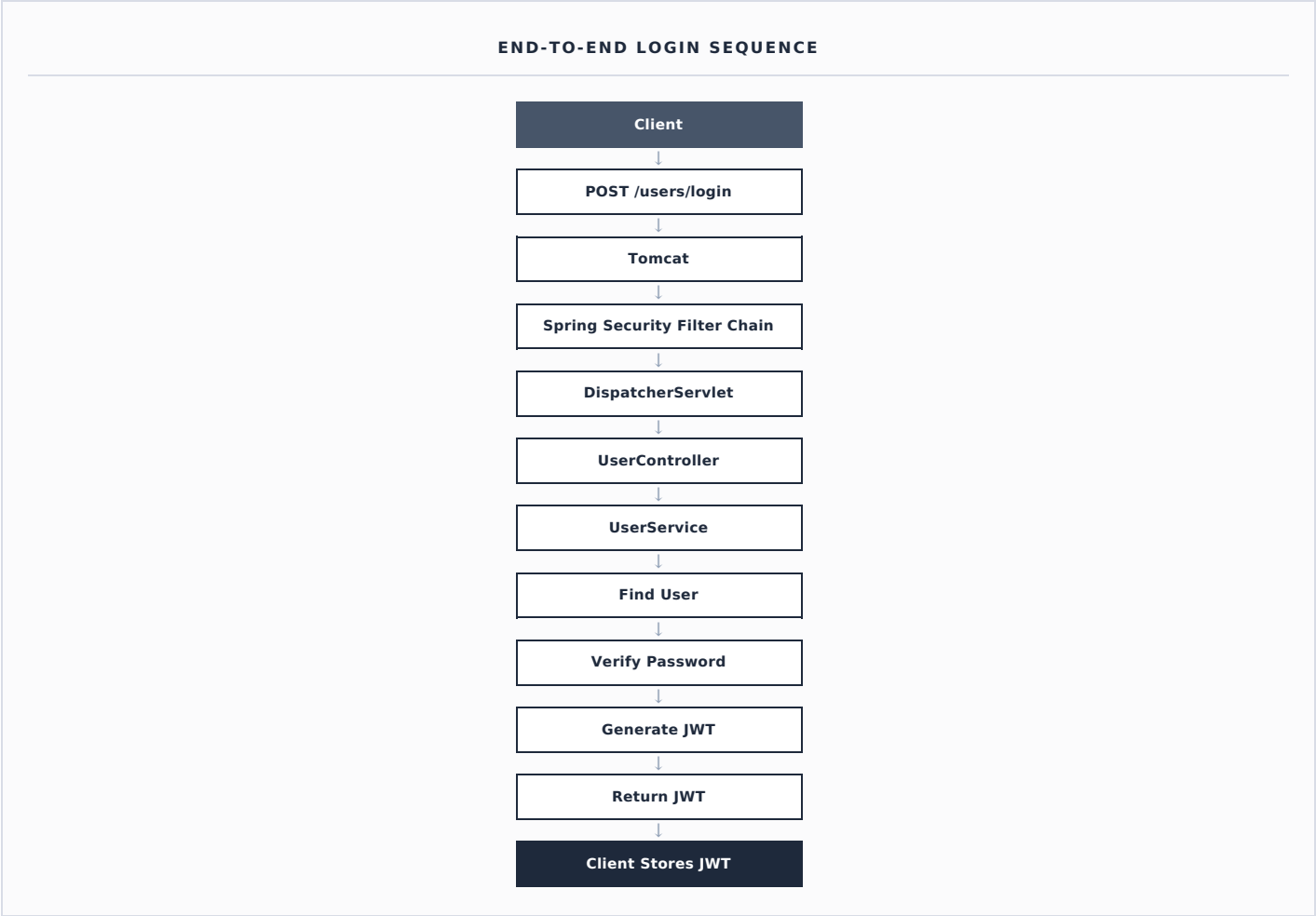
Why Do We Use JWT Instead of Sessions?

Traditional web applications usually create a session after login. The server stores session information in memory, and every request contains a Session ID — the server must remember every logged-in user.

Our application uses JWT instead. After login, the server creates the JWT, the client stores it, every future request includes it, and the server validates the JWT but does **not** store any session information. This approach is called **Stateless Authentication**.

SESSION AUTHENTICATION	JWT AUTHENTICATION
Server stores session data	Client stores JWT
Requires server memory	No server-side session
Session ID identifies user	JWT identifies user
Harder to scale across servers	Easier to scale
Common in traditional MVC apps	Common in REST APIs

Complete Login Flow



Concepts Learned

Authentication	Login Flow	Stateless Authentication	Session Authentication	JWT Generation	Password Verification
BCrypt matches()	Business Logic	Service Layer	Controller Layer		

COMMON MISTAKES

- 1

Comparing Passwords Using equals()

Never compare the entered password directly with the stored password — the stored password is encrypted. Always use:

```
passwordEncoder.matches(rawPassword, hashedPassword);
```
- 2

Generating JWT Before Verification

A JWT should only be created **after** the user's identity has been successfully verified. Generating a token before authentication is a serious security flaw.
- 3

Returning the Password

The login response should return the JWT. The password should never be sent back to the client.
- 4

Confusing Authentication with Authorization

Authentication answers “Who are you?” while Authorization answers “What are you allowed to do?” Login only performs authentication — authorization will be implemented later using user roles.

CHAPTER SUMMARY

In this chapter, we implemented the user login feature. The backend receives the user's email and password, verifies the credentials against the database, and, if authentication succeeds, generates a signed JSON Web Token (JWT).

Unlike traditional session-based systems, our application remains stateless by storing authentication information inside the JWT rather than on the server. This token will now be used in every protected request, where Spring Security and our custom `JwtFilter` will validate it before allowing access to secured resources.